COMP2007: Operating Systems & Concurrency
Week 9 – 12:00pm Monday – 18 November 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Figure: Attendance Monitoring

Operating Systems and Concurrency

Memory Management 6

COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

©University of Nottingham

Inverted Page Table

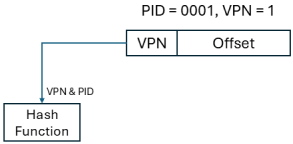
Clarification

PID = 0001, VPN = 1

VPN	Offset

Inverted Page Table

Clarification



Inverted Page Table

Clarification

PID = 0001, VPN = 1

VPN	Offset

VPN & PID

Hash Function

Frame Number	PID	VPN	Protection	Link
0	0789	1	RWX	2
1	0789	2	RWX	null
2	0001	1	RWX	null
3	0789	3	R	null
4	0789	4	R	null
...	...	...	...	null
...	...	...	...	null
N	...	...	...	null

©University of Nottingham

Inverted Page Table

Clarification

PID = 0001, VPN = 1

VPN	Offset

VPN & PID

Hash Function

Frame Number	PID	VPN	Protection	Link
0	0789	1	RWX	2
1	0789	2	RWX	null
2	0001	1	RWX	null
3	0789	3	R	null
4	0789	4	R	null
...	...	...	...	null
...	...	...	...	null
N	...	...	...	null

Inverted Page Table

Clarification

PID = 0001, VPN = 1

VPN	Offset

VPN & PID

Hash Function

Frame Number	PID	VPN	Protection	Link
0	0789	1	RWX	2
1	0789	2	RWX	null
2	0001	1	RWX	null
3	0789	3	R	null
4	0789	4	R	null
...	...	...	...	null
...	...	...	...	null
N	...	...	...	null

PFN	Offset

Inverted Page Table

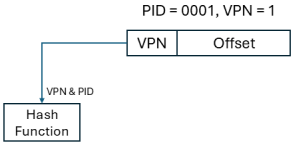
Clarification

PID = 0001, VPN = 1

VPN	Offset

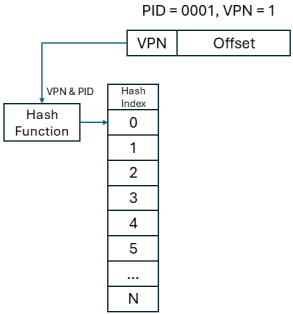
Inverted Page Table

Clarification



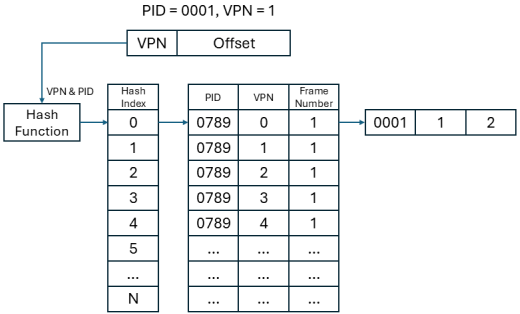
Inverted Page Table

Clarification



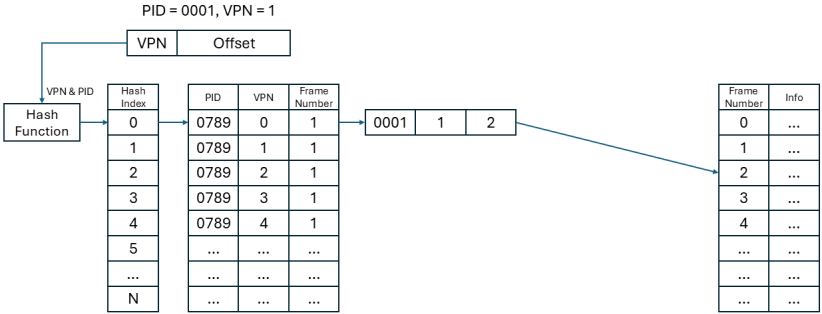
Inverted Page Table

Clarification



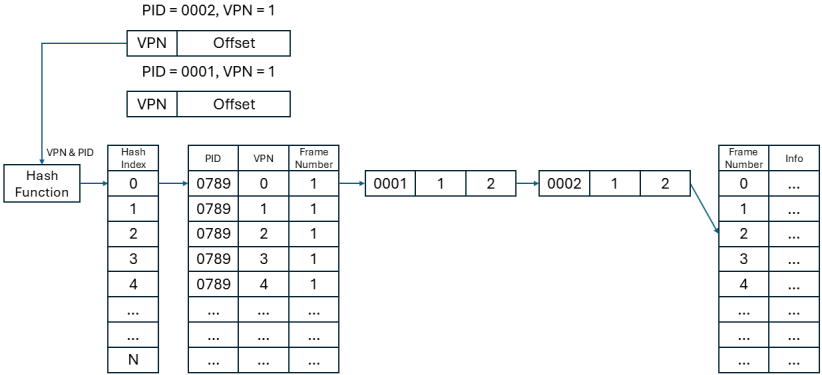
Inverted Page Table

Clarification



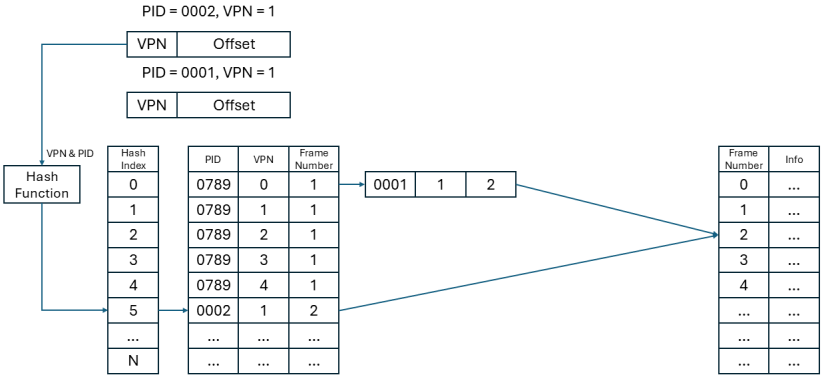
Inverted Page Table

Clarification



Inverted Page Table

Clarification



Goals for Today

Overview

- Several **key decisions** have to be made when **using virtual memory**
  - When are pages **loaded** into memory
  - **What pages** are **removed** from memory ⇒ page **replacement algorithms**
  - **How many pages** are allocated to a process and are they **local or global**
  - **When** are pages **removed** from memory ⇒ paging **daemons**
- What **problems** may occur in virtual memory ⇒ **thrashing**

Page Replacement

Algorithms

1. **Optimal** page replacement
2. **FIFO** page replacement
   - Second chance replacement
   - Clock replacement
3. **Not recently used** (NRU)
4. **Least recently used** (LRU)

Page Replacement

The Clock Replacement Algorithm

- **Second-chance FIFO** can be improved by **maintaining a circular list**
  - A **pointer** points to the last page“visited”
  - The algorithm is called (one-handed) **clock**
  - It is **slow for long lists**
- The **time spent** on maintaining the list **is reduced**

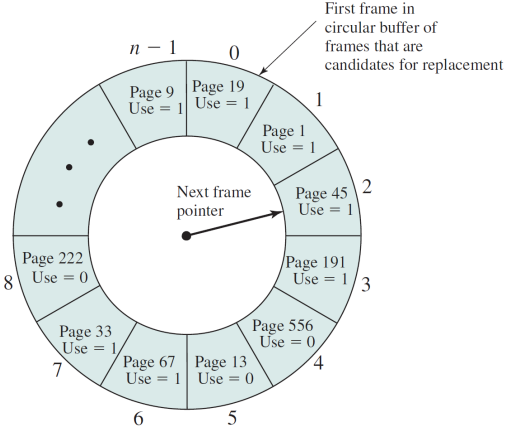


Figure: Clock Replacement Algorithm (Stallings)

Not Recently Used (NRU)

Account for Modified Pages

- **Referenced** and **modified** bits are kept in the page table
  - Referenced bits are set to 0 at the start, and **reset periodically** (e.g. system clock interrupt or when searching the list)
- Four different **page "types"** exist
  - class 0: not referenced and not modified
  - class 1: not referenced and modified
  - class 2: referenced and not modified
  - class 3: referenced and modified

Page Replacement

Not Recently Used (NRU, Cont'ed)

- **Page table entries** are inspected upon every **page fault**
- Can be implemented as:
  1. Find a page from **class 0** to be removed
  2. If step 1 fails, scan again looking for **class 1** and **set the reference bit to 0** for each page visited
  3. If step 2 fails, start again from step 1 (elements from class 2 and 3 have now become class 0 or 1)
- The NRU algorithm provides **good performance** and is **easy to understand and implement**

Page Replacement

Least-Recently-Used

- Least recently used **evicts the page** that has **not been used the longest**
  - The OS **keeps track** of when a page was **last used**
  - Every **page table entry** contains a **field for the counter**
  - **Costly implementation** since it requires a **list of pages sorted** in the order in which they have been used
- The algorithm can be **implemented in hardware** using a **counter** that is incremented after each instruction

©University of Nottingham

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  `0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4`
- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1																								
PF2																								
PF3																								
PF4																								

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  `0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4`
- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0																							
PF2	-																							
PF3	-																							
PF4	-																							

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0																						
PF2	-	2																						
PF3	-	-																						
PF4	-	-																						

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  `0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4`
- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0																					
PF2	-	2	2																					
PF3	-	-	1																					
PF4	-	-	-																					

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0																				
PF2	-	2	2	2																				
PF3	-	-	1	1																				
PF4	-	-	-	3																				

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5																			
PF2	-	2	2	2	2																			
PF3	-	-	1	1	1																			
PF4	-	-	-	3	3																			

Figure: Least Recently Used

©University of Nottingham

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5																		
PF2	-	2	2	2	2	4																		
PF3	-	-	1	1	1	1																		
PF4	-	-	-	3	3	3																		

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5																	
PF2	-	2	2	2	2	4	4																	
PF3	-	-	1	1	1	1	6																	
PF4	-	-	-	3	3	3	3																	

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5																
PF2	-	2	2	2	2	4	4	4																
PF3	-	-	1	1	1	1	6	6																
PF4	-	-	-	3	3	3	3	3																

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7															
PF2	-	2	2	2	2	4	4	4	4															
PF3	-	-	1	1	1	1	6	6	6															
PF4	-	-	-	3	3	3	3	3	3															

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7											
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4											
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6											
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3											

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7	7										
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	4										
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	5										
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3	3										

Figure: Least Recently Used

©University of Nottingham

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7	7	7	7								
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	4	4	4								
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	5	5	5								
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3								

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7	7	7	7	7							
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	4	4	4	1							
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	5	5	5	5							
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3							

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7	7	7	7	7	7	7	7				
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	4	4	4	1	1	1	1				
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	5	5	5	5	5	5	5				
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3				

Figure: Least Recently Used

©University of Nottingham

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7	7	7	7	7	7	7	7	7			
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	4	4	4	1	1	1	1	1			
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	5	5	5	5	5	5	5	2			
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3			

Figure: Least Recently Used

©University of Nottingham

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4

- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	4	4	4	1	1	1	1	1	1	1	
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	5	5	5	5	5	5	5	2	2	2	
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	

Figure: Least Recently Used

Page Replacement

Least-Recently-Used

- Assume we have a system with eight logical address pages & four physical page frames
- Consider the following **page references** in order:

  `0 2 1 3 5 4 6 3 7 4 7 3 3 5 5 3 1 1 1 7 2 3 1 4`
- The number of **page faults** that are generated is **12**

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1	0	0	0	0	5	5	5	5	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	4
PF2	-	2	2	2	2	4	4	4	4	4	4	4	4	4	4	4	1	1	1	1	1	1	1	1
PF3	-	-	1	1	1	1	6	6	6	6	6	6	6	5	5	5	5	5	5	5	2	2	2	2
PF4	-	-	-	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3

Figure: Least Recently Used

Page Replacement Algorithms

Summary

1. **Optimal** page replacement: optimal but not practical
2. **FIFO** page replacement: poor performance but easy to implement
   - Second chance replacement: improved performance but poor implementation
   - Clock replacement: improved implementation but can still be slow
3. **Not recently used** (NRU): easy to understand, moderately efficient (approximation of LRU)
4. **Least recently used** (LRU): close to optimal but more difficult to implement

©University of Nottingham

Resident Set

Size of the Resident Set

- How many pages should be allocated to individual processes:
  - **Small resident sets** enable to store **more processes** in memory ⇒ improved CPU utilisation
  - **Small resident sets** may result in **more page faults**
  - **Large resident sets** may **no longer reduce** the **page fault rate** (**diminishing returns**)
- A trade-off exists between the **sizes of the resident sets** and **system utilisation**
- Resident set sizes may be **fixed** or **variable** (i.e. adjusted at runtime)

Working Sets

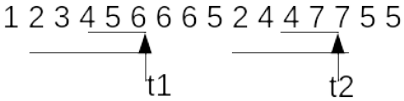
Defining and Monitoring Working Sets

- The **resident set** comprises the set of pages of the process that are in memory
- The **working set** $W(t, k)$ comprises the set **referenced pages** in the last k (= working set window) virtual time units for the process
- k can be defined as "**memory references**" or as "**actual process time**"
  - The set of most recently used pages
  - The set of pages used within a pre-specified time interval

Working Sets

Monitoring Working Sets: Example

- Consider the following page **references in order**:



- If $k = 3$:
  - At t_1, $W(t_1, 3) = \{4, 5, 6\}$
  - At t_2, $W(t_1, 3) = \{4, 7\}$
- If $k = 5$:
  - At t_1, $W(t_1, 5) = \{2, 3, 4, 5, 6\}$
  - At t_2, $W(t_1, 5) = \{2, 4, 7\}$

Working Sets

Defining and Monitoring Working Sets

- The working set is a **function of time** t:
  - Processes **move between localities**, hence, the pages that are included in the working set **change over time**
  - **Stable** intervals alternate with intervals of **rapid change**
- $|W(t,k)|$ is then variable in time. Specifically:

$$1 \leq |W(t,k)| \leq min(k,N) \tag{1}$$

where N is the total number of pages of the process.

©University of Nottingham

Working Sets

Monitoring Working Sets

- Choosing the right value for k is paramount:
  - Too **small**: inaccurate, pages are missing
  - Too **large**: too many unused pages present
  - **Infinity**: all pages of the process are in the working set
- **Working sets** can be used to guide the **size of the resident sets**
  - Monitor the working set
  - Remove pages from the resident set that are not in the working set
- The working set is costly to maintain $\Rightarrow$ **page fault frequency (PFF)** can be used as an approximation
  - If the PFF is increased -> we need to increase
  - If PFF is very reduced -> we may try to decrease

Resident Set

Size of the Resident Set

- How many pages should be allocated to individual processes:
  - **Small resident sets** enable to store **more processes** in memory $\Rightarrow$ improved CPU utilisation
  - **Small resident sets** may result in **more page faults**
  - **Large resident sets** may **no longer reduce** the **page fault rate** (**diminishing returns**)
- A trade-off exists between the **sizes of the resident sets** and **system utilisation**

Resident Set

Size of the Resident Set

- For **variable sized** resident sets, **replacement policies** can be:
  - **Local**: a page of the same process is replaced
  - **Global**: a page can be taken away from a different process
- Variable sized sets require **careful evaluation of their size** when a **local scope** is used (based on the **working set** or the **page fault frequency**)

Resident Sets

Local vs. Global Replacement

- **Global replacement policies** can select frames from the entire set, i.e., they can be “taken” from other processes
  - Frames are **allocated dynamically** to processes
  - Processes cannot control their own page fault frequency, i.e., the **PFF** of one process is **influenced by other processes**
- **Local replacement policies** can only select frames that are allocated to the current process
  - Every process has a **fixed fraction of memory**
  - The locally **“oldest page”** is not necessarily the globally “oldest page”
- **Page replacements algorithms** explained before can use both policies.
  - Windows uses a **variable approach** with **local replacement**

Resident Set

Local vs. Global approaches

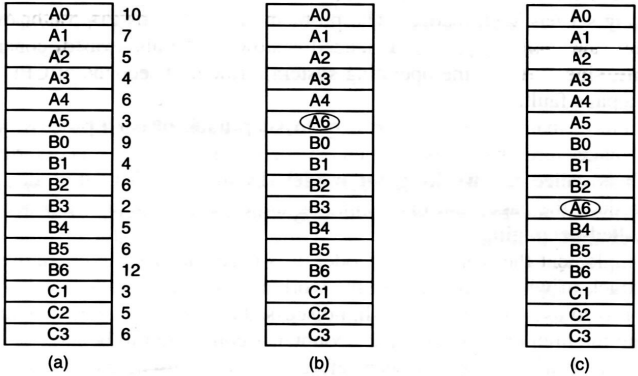


Figure: Local vs. global page replacement. (a) Original config, number at the right represents loading time (b) Local (c) Global (Tanenbaum)

Paging Daemon

Pre-cleaning (⇔ demand-cleaning)

- It is more efficient to **proactively** keep a number of **free pages** for **future pa**
  - If not, we may have to **find a page** to evict and **write it to the drive**
  - This can increase the **page fault time**
- Many systems have a background process called a **paging daemon**
  - Runs at **periodic intervals**
  - Selects **pages to evict** (using page replacement algorithms) if too few frames are free

Paging Daemon

Pre-cleaning ($\Leftrightarrow$ demand-cleaning)

- It is more efficient to **proactively** keep a number of **free pages** for **future page faults**
  - If not, we may have to **find a page** to evict and **write it to the drive**
  - This can increase the **page fault time**
- Many systems have a background process called a **paging daemon**
  - Runs at **periodic intervals**
  - Selects **pages to evict** (using page replacement algorithms) if too few frames are free
- Paging daemons can be combined with **buffering** (free and modified lists) $\Rightarrow$ write the modified pages **but keep them in main memory** when possible

Thrashing

Defining Thrashing

- Assume **all available pages are in active use** and a new page needs to be loaded:
  - The page that will be **evicted** will have to be **reloaded soon afterwards** (it is still active)
- **Thrashing** occurs when pieces are swapped out and loaded again immediately

Thrashing

A Vicious Circle?

- CPU utilisation is too low ⇒ scheduler **increases degree of multi-programming**
  - ⇒ Frames are allocated to new processes and **taken away from existing processes**
    - ⇒ I/O **requests are queued** up as a consequence of page faults
- CPU **utilisation drops further** ⇒ scheduler increases degree of multi-programming

Thrashing

Causes/Solutions

- **Causes** of thrashing include:
  - The degree of multi-programming is too high, i.e., the total **demand** (i.e., the sum of all **working set** sizes) **exceeds supply** (i.e. the available frames)
  - An individual process is allocated **too few pages**
- This can be **prevented** by, e.g., using good **page replacement policies**, reducing the **degree of multi-programming** (medium term scheduler), or adding more memory
- The **page fault frequency** can be used to detect that a system is thrashing

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame:

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame: 12 (2^{12} addresses)
  - How many bits are used for the page number:

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame: 12 (2^{12} addresses)
  - How many bits are used for the page number: 4 (2^4 pages)
  - How many bits are used for the frame number:

Test Your Understanding

Address Translation: Exercises

- Given a 4KB page/frame size, and a 16-bit address space:
  - How many bits are used for the offset within the page/frame: 12 (2^{12} addresses)
  - How many bits are used for the page number: 4 (2^4 pages)
  - How many bits are used for the frame number: $\leq$ 4/not specified

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0?

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0?
  - Page index 0 offset 0 $\Rightarrow$ frame index 2 offset 0
  - 2 * 4096 = 8192

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0?
  - Page index 0 offset 0 $\Rightarrow$ frame index 2 offset 0
  - 2 * 4096 = 8192
- What is the physical address for logical address 8192?
  - Page index 2 offset 0 $\Rightarrow$ frame with index 6 offset 0
  - 6 * 4096 = 24576

©University of Nottingham

Test Your Understanding

Address Translation: Exercises (Cont'ed)

Pages		Frames	
0	0000	0010	2
1	0001	0001	1
2	0010	0110	6
3	0011	0000	0
4	0100	0100	4
5	0101	0011	3
6	0110	X	X
7	0111	X	X
8	1000	X	X
9	1001	0101	5
10	1010	X	X
11	1011	0111	7
12	1100	X	X
...	...	...	...

Table: Page table

- What is the physical address for logical address 0?
  - Page index 0 offset 0 $\Rightarrow$ frame index 2 offset 0
  - 2 * 4096 = 8192
- What is the physical address for logical address 8192?
  - Page index 2 offset 0 $\Rightarrow$ frame with index 6 offset 0
  - 6 * 4096 = 24576
- What is the physical address for logical address 20500?
  - Page index 5 offset 20 $\Rightarrow$ frame with index 3 offset 20
  - 3 * 4096 + 20 => 12308

Test Your Understanding

FIFO vs. optimal page replacement

- Compare FIFO/LRU with **the optimal page replacement** algorithm. The process starts up with none of its pages in memory.
- What would be the minimum number of **page faults** that would be generated by the optimal approach?

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	2	3	1	4
PF1																								
PF2																								
PF3																								
PF4																								

Figure: Optimal Page Replacement

©University of Nottingham

Summary

Take-Home Message

- Second Chance FIFO, Clock Replacement, NRU, LRU page replacement
- Page allocations to processes (variable, fixed, local, global)
- Page Daemons
- Thrashing
- Reading: Tanenbaum Section 3.4, 3.5.1, 3.5.8

©University of Nottingham